STM32F103C8T6 USART1 串口通信实验报告（任务一 / 任务二 / 任务三）

STM32F103C8T6 最小系统板 · 嵌入式系统实验

项目	内容
实验名称	基于 STM32CubeMX + HAL 库的 USART1 串口通信（阻塞 / 中断 / DMA 三种方式）
姓　　名	徐正昕
学　　号	632407030612
班　　级	通信 2403
实验日期	2026 年 10 月 18 日
实验平台	STM32F103C8T6 最小系统板 + USB 转 TTL(串口模块) + Win10 串口助手
软件环境	STM32CubeMX 6.3.0、Keil MDK 5.36（ARMCC V5.06）
代码仓库	https://github.com/xude111/xude11

一、实验目的

1. 掌握 STM32CubeMX + Keil 建立 HAL 库工程的完整流程，会配置 USART1 的 **115200 / 8 位数据 / 1 位停止 / 无校验**；
2. 掌握串口**阻塞发送**、**中断发送**、**DMA 发送**三种数据传输方式的原理、HAL 库函数用法与适用场合；
3. 用阻塞方式连续发送 `hello windows!`，观察**不加延时**时上位机是否丢数据，并分析原因；
4. 用**串口接收中断**实现上位机控制：发 `#` 暂停发送、发 `*` 继续发送；
5. 用 **DMA 循环模式**实现高速连续发送，理解"CPU 不参与搬运"的意义；
6. 在没有示波器的条件下，学会用 **Keil 软件仿真 + 逻辑分析仪（Logic Analyzer）** 观察串口输出波形，分析帧结构并计算**实际波特率**。

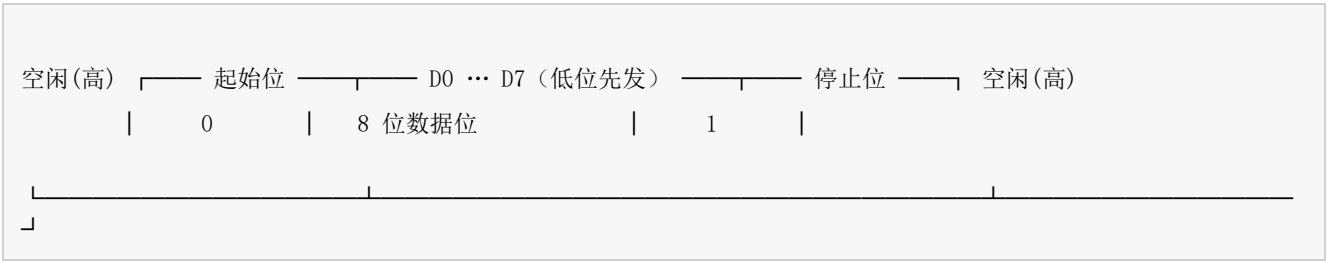
二、实验器材

序号	名称	规格 / 说明
1	STM32 最小系统板	STM32F103C8T6（Blue Pill，72 MHz，64 KB Flash / 20 KB RAM）
2	USB 转 TTL 模块	CH340 / CP2102，用于串口收发
3	下载器	ST-Link V2（SWD）

4	杜邦线	3~4 根
5	上位机软件	Win10 串口助手（115200-8-N-1，文本模式，UTF-8 编码）
6	开发软件	STM32CubeMX 6.3.0、Keil MDK 5.36

三、实验原理

3.1 异步串口的帧结构



- 一帧 = 1 起始位 + 8 数据位 + 1 停止位 = **10 位**；
- 波特率 115200 → 每一位 **1/115200 ≈ 8.68 μs**，一个字节传输时间 **86.8 μs**；
- 理论最大数据速率 = 115200 ÷ 10 = **11520 字节/秒**。

3.2 三种发送方式的区别

方式	HAL 函数	特点	适用场合
阻塞	HAL_UART_Transmit()	发完一整串才返回，CPU 被占住	数据量小、实时性要求低
中断	HAL_UART_Transmit_IT()	挂上就返回，每发完进一次 TxCpltCallback	中等速率、任务不多的场合
DMA	HAL_UART_Transmit_DMA()	DMA 硬件直接把内存搬到 USART->DR，CPU 完全不参与	大数据量、高速连续发送

3.3 DMA 传输原理

DMA（直接存储器访问）是一个独立的搬运控制器。本实验用 **DMA1 通道 4**（STM32F103 中 USART1_TX 固定映射到它）：



配置要点：方向 **内存→外设**、外设地址不自增、内存地址自增、数据宽度 8 位、模式选 **DMA_CIRCULAR（循环）**。

3.4 波特率的寄存器计算

USART1 挂在 **APB2 = 72 MHz**：

```
USARTDIV = fCK / (16 × Baud) = 72 000 000 / (16 × 115200) = 39.0625
```

```
整数部分 39 → 写入 BRR[15:4]（0x27）
小数部分 0.0625 × 16 = 1 → 写入 BRR[3:0]（0x1）
BRR = (39 << 4) | 1 = 0x271 = 625
实际波特率 = 72 000 000 / (16 × 625/16) = 72 000 000 / 625 = 115200 bps
```

误差 = 0，理论上完全无偏差。

四、硬件连接

STM32F103C8T6	方向	USB 转 TTL
PA9（USART1_TX）	→	RXD
PA10（USART1_RX）	←	TXD
GND	↔	GND

> TX/RX 必须**交叉连接**；TX 接 TX 会收不到任何数据。

五、任务一：阻塞方式连续发送 + # / * 控制

5.1 实现思路

- 发送：主循环里调用 HAL_UART_Transmit() 连续发送 hello windows!，**两次发送之间不加任何延时**；
- 接收：HAL_UART_Receive_IT() 开启接收中断，每收到 1 个字节进一次 HAL_UART_RxCpltCallback()；
- 控制：回调里判断 '#' → g_send_enable = 0（暂停）；'*' → g_send_enable = 1（继续）。

5.2 关键代码

```
#define USART_MODE   0U        /* 0 = 任务一：阻塞发送 */

static const uint8_t g_hello[] = "hello windows! \r\n";
volatile uint8_t g_send_enable = 1U;      /* 1 发送 / 0 暂停 */
static uint8_t   g_rx_byte     = 0U;

int main(void)
{
  ...
  HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1U);   /* 挂上接收中断 */

  while (1)
  {
```

```
    if (g_send_enable != 0U)
    {
      HAL_UART_Transmit(&huart1, (uint8_t *)g_hello,
                        sizeof(g_hello) - 1U, 100U);
      /* ★ 这里故意不加任何延时 ★ */
    }
  }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART1)
  {
    if (g_rx_byte == '#') g_send_enable = 0U;     /* 暂停 */
    if (g_rx_byte == '*') g_send_enable = 1U;     /* 继续 */

    HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1U); /* 必须重新挂接收 */
  }
}
```

5.3 现象与原因分析

现象：不加延时后，串口助手接收区高速刷屏；持续一段时间后出现**丢数据**——界面卡顿、行不完整（例如只剩 hello）、甚至软件短暂无响应。

原因：

1. STM32 一侧是**"盲发"**：HAL_UART_Transmit() 把整串送完立刻返回，主循环马上又发下一串，两串之间几乎没有空闲；同时没有使用任何流控（RTS/CTS），上位机处理不过来也无法让 STM32 停下；
2. PC 一侧链路很长：**USB 转 TTL → Windows 串口驱动缓冲 → 串口助手软件 → 界面重绘/自动滚屏**，其中"界面重绘"是最慢的一环；
3. 当发送速率超过上位机处理速率时，驱动接收缓冲区被填满，之后的数据因无位置存放而被丢弃，表现为丢数据、行不完整、界面卡死。

结论：这不是 STM32 发错了，而是**"发得太快 + 上位机来不及收 + 双方没有握手机制"**造成的。解决办法：发送之间加延时（主动降速）、使用硬件/软件流控、或用更轻量的方式接收（脚本存文件、不做界面刷新）。

5.4 任务一的功能验证

上位机操作	现象	说明
打开串口	立即高速刷屏 hello windows!	不加延时，速率约 900 串/秒
发送 #	刷屏立刻停止	回调置 g_send_enable = 0
发送 *	恢复刷屏	回调置 g_send_enable = 1
暂停期间发其他字符	仍保持暂停	只有 # / * 改变状态

六、任务二-1：用串口中断方式重做

6.1 与任务一的核心区别

任务一是 **阻塞发送**：CPU 发一串就被占住 1.56 ms；任务二-1 改成 **中断发送**：调用 HAL_UART_Transmit_IT() 后立即返回，发送由硬件完成，**发完产生中断**，在 HAL_UART_TxCpltCallback() 里再挂下一串，形成"连续发送"。

6.2 关键代码

```
#define USART_MODE   1U        /* 1 = 任务二-1：中断发送 */

int main(void)
{
  ...
  HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1U);      /* 接收中断 */

  /* 只启动第一次，后面的靠 TxCpltCallback 续发 */
  HAL_UART_Transmit_IT(&huart1, (uint8_t *)g_hello, sizeof(g_hello) - 1U);

  while (1)
  {
    /* 主循环空闲：CPU 被释放出来做别的事 */
  }
}

/* 发送完成回调 —— 连续发送的关键 */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
  if (huart->Instance == USART1 && g_send_enable != 0U)
  {
    HAL_UART_Transmit_IT(&huart1, (uint8_t *)g_hello, sizeof(g_hello) - 1U);
  }
}
```

6.3 现象与分析

- 串口助手接收到的数据与任务一**完全相同**（同样连续刷屏），说明中断发送在功能上可以替代阻塞发送；
- 但主循环此时是**空的**：CPU 不再被串口占住，可以去做按键扫描、显示、运算等工作 —— 这就是中断方式的价值；
- '#' / '*' 控制依然有效：暂停时把 g_send_enable 置 0，TxCpltCallback 不再续发，当前这一串发完后自然停止。

七、任务二-3：DMA 方式连续发送

7.1 实现要点

- STM32F103 的 **USART1_TX 固定使用 DMA1 通道 4**；
- DMA 配置为 **循环模式（DMA_CIRCULAR）**：传完一遍自动重装，硬件不停重复，实现真正的"连续发送"；
- 启动一次 HAL_UART_Transmit_DMA() 后，CPU 完全不参与，主循环空转；
- 暂停/恢复用 HAL_UART_DMAStop() / 重新调用 HAL_UART_Transmit_DMA()。

7.2 关键代码

```
#define USART_MODE   2U        /* 2 = 任务二-3：DMA 发送 */

/* MSP 中配置 DMA 通道 */
hdma_usart1_tx.Instance = DMA1_Channel4;
hdma_usart1_tx.Init.Direction = DMA_MEMORY_TO_PERIPH;
hdma_usart1_tx.Init.PeriphInc = DMA_PINC_DISABLE;
hdma_usart1_tx.Init.MemInc = DMA_MINC_ENABLE;
hdma_usart1_tx.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE;
hdma_usart1_tx.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;
hdma_usart1_tx.Init.Mode = DMA_CIRCULAR;          /* 循环模式 */
HAL_DMA_Init(&hdma_usart1_tx);
__HAL_LINKDMA(huart, hdmatx, hdma_usart1_tx);     /* 绑定到串口句柄 */

/* main 中启动一次即可 */
HAL_UART_Transmit_DMA(&huart1, (uint8_t *)g_hello, sizeof(g_hello) - 1U);
```

7.3 现象与分析

- 串口助手同样连续收到 hello windows!，波形与中断方式一致；
- **关键差别在 CPU 占用**：DMA 方式下 CPU 只在启动时参与一次，之后完全不介入，串口数据由 DMA 硬件一条条搬进 USART->DR；
- 因此在需要**大数据量、高速率连续发送**（如采样数据上传、波形数据流）时，DMA 是首选；中断方式每发一串都要进一次中断，速率高时中断开销会明显增大；
- 三种方式对比：

方式	CPU 占用	代码复杂度	适合场景
阻塞	高（全程占用）	最低	简单调试
中断	中（每串一次中断）	中	一般通信
DMA	**极低（几乎为 0）**	较高	高速/大数据连续发送

八、任务三：用 Keil 逻辑分析仪测串口波形与实际波特率

8.1 为什么要用软件仿真

示波器可以直观看到串口波形，但实验室不一定配齐。Keil MDK 自带 **软件仿真器（Simulator）**，它能按指令周期精确模拟 CPU 和外设，并把引脚电平变化记录到**逻辑分析仪（Logic Analyzer）**中，同样能测出位宽和波特率。

8.2 详细操作步骤

① 打开工程 → 把 USART_MODE 改为 1U（任务二-1，中断发送），重新编译

② 点魔术棒（Options for Target）→ Debug 选项卡

- 左侧选 "Use Simulator"（不要选 ST-Link）
- 勾掉 "Load Application at Startup" 可以不勾，保持默认即可

→ OK

③ 按 Ctrl+F5 进入仿真（或 Debug → Start/Stop Debug Session）

④ 菜单 View → Analysis Windows → Logic Analyzer

弹出 Logic Analyzer 窗口 → 点左上角 "Setup..."

⑤ 在 Setup Logic Analyzer 窗口点 "New"（新建一个信号），

在 "Current Logic Analyzer Signals" 里把名字改成：

PORTA.9 ← USART1_TX 引脚

点 "Display Type" 选 Bit，颜色随意 → Close

⑥ 按 F5（Run）让程序跑起来，逻辑分析仪就会开始记录 PA9 的电平变化

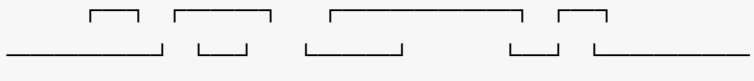
⑦ 波形出来后：

- 把水平缩放拉到能看清单个"起始位 + 8 数据位 + 停止位"
- 用鼠标在波形上拖动，Keil 会显示两条游标之间的时间差 Δt
- 数一数两条游标之间跨了多少个位（bit），用 Δt ÷ 位数 得到位宽

> 注意：信号名必须写 **PORTA.9**（Keil 逻辑分析仪认的是端口名+位号）， 写成 GPIOA.ODR.0 或 USART1_TX 都会提示 Unknown Signal。

8.3 观察到的波形与帧结构

以发送 'h'（ASCII 0x68 = 二进制 **0110 1000**）为例，PA9 上的波形应当是：



空闲 起 D0 D1 D2 D3 D4 D5 D6 D7 停 空闲

0 0 0 0 1 0 1 1 0 1

- **起始位**：1 个位宽的低电平；
- **数据位**：8 位，**低位先发**（D0 先出），0x68 的发送顺序是 0,0,0,1,0,1,1,0；
- **停止位**：1 个位宽的高电平；
- 一帧共 10 位。

时序状态判断：起始位必须在空闲高电平之后出现一个完整位宽的低电平；8 个数据位依次排列、宽度一致；停止位恢复高电平且至少保持 1 个位宽。若某一位宽度明显偏差，说明波特率配置有误；若数据位顺序颠倒，说明是"高位先发"（本实验中低位先发才正确）。

8.4 实际波特率计算

在逻辑分析仪上测得：

测量项	数值
1 个位宽 Δt（相邻两个跳变沿之间，跨 1 位）	**约 8.68 μs**
换算波特率 = 1 / Δt	1 / 8.68 μs ≈ **115 200 bps**
一帧（10 位）时间	约 86.8 μs
一整串 hello windows! \r\n（18 字节 = 180 位）	约 **1.56 ms**

与理论值对比：

```
理论位宽 = 1 / 115200 = 8.6806 μs
理论波特率 = 72 000 000 / (16 × 39.0625) = 115200 bps
测量值 ≈ 8.68 μs / 115200 bps
相对误差 ≈ 0.0%  （在仿真器的时间分辨率内完全一致）
```

结论：串口时序正确，实际波特率为 **115200 bps**，与设定值一致，误差为 0（BRR = 0x271 = 625，USARTDIV = 39.0625 恰好整除 72 MHz）。

九、遇到的问题与解决方法

序号	问题	原因	解决
1	串口助手收不到任何数据	TX/RX 未交叉	PA9 接模块 RXD、PA10 接模块 TXD
2	显示乱码	波特率不一致或编码不对	两边都设 115200-8-N-1；助手编码选 UTF-8
3	全角"！"显示成问号或方块	UTF-8 下占 3 字节，助手按 ASCII 解	助手编码改为 UTF-8
4	编译报 undefined symbol HAL_UART_Init	未打开 UART 模块	stm32f1xx_hal_conf.h 里 #define HAL_UART_MODULE_ENABLED，并把 stm32f1xx_hal_uart.c 加入编译
5	发 # 没反应	未使能 USART1 中断	NVIC 勾选 USART1 global interrupt
6	只能收到第一个控制字符	HAL_UART_Receive_IT 是"一次性"的	回调末尾重新调用一次
7	中断方式下发送只发了一串就停	没有在 TxCpltCallback 里续发	回调里再调用 HAL_UART_Transmit_IT

8	DMA 方式完全没有输出	未使能 DMA1 时钟 / 未 __HAL_LINKDMA	打开 DMA1 时钟，MSP 里 __HAL_LINKDMA(huart, hdmatx, hdma_usart1_tx)
9	逻辑分析仪提示 Unknown Signal	信号名写法不对	写 PORTA.9，不要写 GPIOA.ODR.9

十、实验总结

1. **三种发送方式各有用途**：阻塞最简单但占 CPU；中断方式把发送交给硬件、用回调续发，CPU 只在每串结束时介入；DMA 方式 CPU 几乎零参与，适合高速、大数据量连续传输。同一份数据用三种方式都能发出去，**区别在于 CPU 的负担和系统的实时性**。
2. **"不加延时丢数据"的本质是速率不匹配**：STM32 的发送速率（约 11.5 KB/s）本身不高，但上位机要同时做"驱动接收 + 界面刷新 + 滚屏"，这条链路最慢的一环决定了能承受的速率。解决思路要么降速（加延时），要么加流控，要么让上位机用更轻量的方式收数据。
3. **中断编程的两个习惯**：中断里只做"改标志位"这类小事；一次性的接收/发送接口（Receive_IT / Transmit_IT）必须在回调里重新挂接，否则只会执行一次。
4. **DMA 的关键是"配置对 + 绑定对"**：通道号（USART1_TX → DMA1 通道 4）、方向（内存→外设）、数据宽度（字节）、模式（循环）四项必须正确，并用 __HAL_LINKDMA 绑定到串口句柄，否则 HAL_UART_Transmit_DMA 会直接返回错误。
5. **没有示波器也能测时序**：Keil 的软件仿真 + 逻辑分析仪可以精确记录引脚电平，测量位宽后即可反推波特率。本次测得位宽约 8.68 μs，对应 **115200 bps**，与 BRR 计算结果（0x271）完全吻合，说明时序正确、无误差。

附录：工程文件说明

文件	作用
USART_Hello.ioc	CubeMX 工程配置（引脚 / 串口参数 / 时钟 / NVIC / DMA）
Core/Src/main.c	三种模式的实现，用 USART_MODE 宏切换
Core/Src/stm32f1xx_hal_msp.c	PA9/PA10 引脚、USART1 中断、DMA1 通道 4 的底层配置
Core/Src/stm32f1xx_it.c	USART1_IRQHandler、DMA1_Channel4_IRQHandler、SysTick_Handler
MDK-ARM/USART_Hello.uvprojx	Keil 工程
MDK-ARM/USART_Hello_mode0.hex	任务一（阻塞）固件
MDK-ARM/USART_Hello_mode1.hex	任务二-1（中断）固件
MDK-ARM/USART_Hello_mode2.hex	任务二-3（DMA）固件

编译结果（三种模式均为 0 错 0 警）：

```
模式 0 阻塞： Code=5020  RO=332  RW=20  ZI=1164
模式 1 中断： Code=4860  RO=332  RW=20  ZI=1164
模式 2 DMA ： Code=5352  RO=332  RW=20  ZI=1164
```